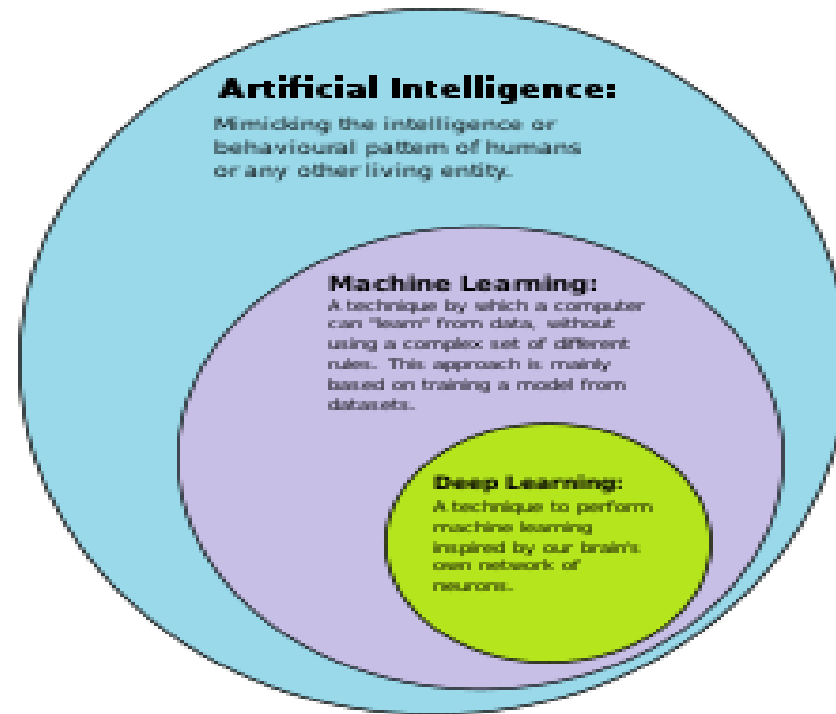


Sección 5: Machine Learning

Contexto IA



¿Qué es el Machine Learning?

- El aprendizaje Automático es un subdominio de la IA que proporciona a los sistemas, la capacidad de aprender y mejorar automáticamente a partir de la experiencia, sin ser programados explícitamente (Arthur Samuel 1959). Se basa en la hipótesis subyacente de crear el modelo y tratar de mejorarlo ajustando más datos en el modelo a lo largo del tiempo.
- Experiencia (datos) -> Modelo -> Predicciones

¿Cuándo utilizar ML?

- En soluciones que funcionan mediante la aplicación de un conjunto extenso de reglas o heurísticas
- En problemas complejos, en los que el analista no es capaz de determinar una solución a partir de la información existente
- En entornos que fluctúan o varían con frecuencia
- Apoyo de la fase de análisis en enfoques tradicionales en los que se dispone de conjuntos de datos muchos más grandes y difíciles de interpretar.

Aprendizaje supervisado

- Utiliza datos etiquetados (pares de entrada-salida) para entrenar modelos
- La regresión predice valores continuos (por ejemplo, precios de viviendas).
- La clasificación o regresión logística, predice valores discretos (por ejemplo, si un mensaje es spam o no).

Aprendizaje no supervisado

- Infiere una función para describir la estructura de datos sin etiquetar, a diferencia del aprendizaje supervisado, que realiza predicciones a partir de datos etiquetados.
- Diferencia clave: El aprendizaje no supervisado se centra en describir la estructura de los datos, no en realizar predicciones.
- Aplicación: Resulta útil para analizar grandes conjuntos de datos sin etiquetar, con el fin de encontrar patrones y obtener información valiosa.

Aprendizaje online y batch

- Aprendizaje por lotes: Se entrena con todos los datos disponibles a la vez, lo que requiere reentrenamiento con nuevos datos. Es más sencillo, pero menos adaptable.
- Aprendizaje en línea: Se entrena de forma incremental con puntos de datos individuales o minilotes, lo que permite la adaptación continua y el manejo de grandes conjuntos de datos. Sin embargo, puede ser inestable con datos de baja calidad y requiere prefiltrado.

Aprendizaje basado en instancias y basado en modelos

- El aprendizaje basado en instancias utiliza medidas de similitud para comparar nuevos datos con ejemplos de entrenamiento almacenados.
- El aprendizaje basado en modelos construye una función de hipótesis (un modelo matemático) para realizar predicciones.

Sección 6: Regresión Lineal

Regresión Lineal: Introducción

- Es un algoritmo de aprendizaje supervisado, basado en modelos, que predice valores continuos.
- El algoritmo construye una función de hipótesis lineal para modelar datos con una tendencia lineal.
- Realiza predicciones computando una suma ponderada de las características de entrada y sumándole una constante conocida como Bias o sesgo.
- Este algoritmo intenta predecir valores continuos, clasificándose dentro de las técnicas de regresión en el aprendizaje supervisado.
- Su objetivo es construir una función hipótesis que se ajuste de la mejor manera posible a la tendencia lineal de los datos de entrenamiento.

- Un ejemplo de su uso es predecir el coste de un incidente de seguridad basándose en el número de sistemas afectados. En este caso, el algoritmo utilizaría datos de incidentes pasados (datos etiquetados) que incluyen el número de equipos afectados (característica de entrada) y el coste que tuvo ese incidente (característica de salida o valor a predecir). La salida del algoritmo sería una función matemática lineal que representa la tendencia de estos datos, permitiendo predecir el coste de futuros incidentes.

- En una regresión lineal, las características de entrada o variables de entrada se denotan siempre con una X. Si hay una única característica de entrada, se representa como X1. Si hay más de una característica de entrada, se utilizan X1, X2, X3 y así sucesivamente hasta XN, donde N es el número de características o variables de entrada.
- La variable de salida se corresponde con la letra Y. Normalmente es una única variable de salida, aunque algunos algoritmos permiten realizar predicciones múltiples, en cuyo caso se usarían Y1, Y2, etc.
- Un ejemplo de entrenamiento se corresponde con la tupla (X, Y), donde X puede ser un conjunto de características de entrada (X1, X2, ..., XN) e Y es el valor de salida

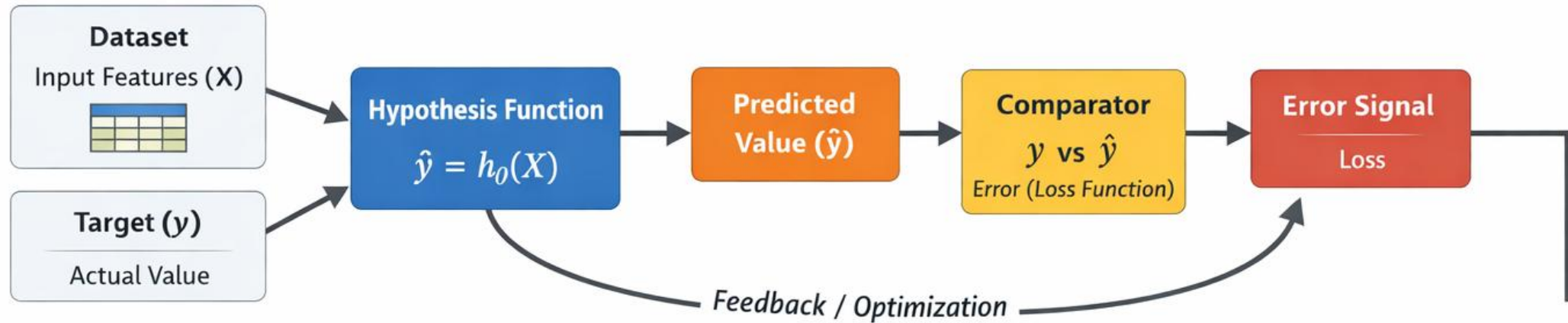
Regresión Lineal: Función hipótesis

- La función hipótesis es la función matemática asociada con la regresión lineal. Esta función se ajusta a la tendencia de los datos del conjunto de datos, dando como resultado un modelo. El modelo, que es la función hipótesis ajustada, se utiliza posteriormente para realizar predicciones.
- En el caso de la regresión lineal univariable, la función hipótesis es la función de la recta, que se representa como $H(x) = \theta_0 + \theta_1 x$. Aquí, θ_0 y θ_1 son los parámetros del modelo. θ_0 determina el punto de corte con el eje Y, y θ_1 indica la pendiente de la recta. El objetivo es encontrar los valores óptimos de θ_0 y θ_1 que ajusten la recta lo mejor posible a la tendencia de los datos.
- Cuando hay múltiples características de entrada (regresión lineal multivariable), la función hipótesis cambia a $H(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$. En este caso, hay más parámetros del modelo ($\theta_0, \theta_1, \theta_2, \dots, \theta_n$), donde θ_0 es el término de sesgo (bias) y $\theta_1, \theta_2, \dots, \theta_n$ son los pesos o ponderaciones que ponderan las características de entrada ($x_1, x_2, \dots, x_n$).
- En resumen, la función hipótesis es una suma ponderada de las entradas, donde los parámetros del modelo (θ) son los pesos que determinan la forma específica de la función matemática que se ajustará a los datos.

Regresión Lineal: Construcción del Modelo

- El principal desafío consiste en encontrar los valores óptimos de θ_0 y θ_1 que mejor se ajusten a los datos de entrenamiento.
- Esto se logra mediante una función de costo para medir los errores de predicción y una función de optimización para minimizarlos; este proceso se denomina entrenamiento.

Diagrama de flujo de aprendizaje supervisado



Proceso general de entrenamiento de un algoritmo de machine learning

1. **Inicialización de parámetros:** Se inicializan los parámetros θ_0 y θ_1 de manera aleatoria.
2. **Cálculo del error con la función de coste:** Se utiliza una función de coste o función de error para calcular el error de la función hipótesis ajustada con los valores aleatorios de los parámetros, en relación con los ejemplos del conjunto de datos. Este cálculo requiere que el conjunto de datos esté etiquetado, es decir, se necesita conocer la variable de salida de los datos.
3. **Minimización del error con la función de optimización:** Se emplea una función de optimización para minimizar este error. Esta función se encarga de modificar los parámetros θ_0 y θ_1 de manera que la función hipótesis se ajuste lo mejor posible a la tendencia de los datos.
4. **Iteración hasta la convergencia:** El proceso de modificar los parámetros y calcular el error continúa hasta que el error que proporciona la función de coste ya no pueda disminuirse más. En ese punto, la función hipótesis se habrá ajustado lo mejor posible a la tendencia de los datos, y se considera que el modelo ha sido construido.

Estrategias de cálculo y actualización

Método	Cómo se calcula el error	Cuándo se actualizan los parámetros	Ventajas	Desventajas
Batch Gradient Descent	Se calcula el error con todas las instancias del dataset y se obtiene un promedio.	Una vez por época (<u>epoch</u>)(al terminar de evaluar todos los datos).	Preciso y estable.	Lento con conjuntos grandes.
Stochastic Gradient Descent (SGD)	Se calcula el error para una sola instancia.	Después de cada instancia.	Rápido y eficiente en datasets grandes.	Actualizaciones ruidosas y menos estables.
Mini-Batch Gradient Descent	Se calcula el error por grupos pequeños (por ejemplo, 32 o 100 instancias).	Una vez por cada mini-batch.	Equilibrado entre precisión y velocidad.	Requiere elegir bien el tamaño del lote.

Esquema resumido

Batch Gradient Descent:

[Datos completos] → Calcular error medio → Actualizar parámetros → Repetir

Stochastic Gradient Descent:

Instancia 1 → Actualizar

Instancia 2 → Actualizar

Instancia 3 → Actualizar

...

→ Repetir

Mini-Batch Gradient Descent:

Lote 1 (ej. 32 muestras) → Calcular error medio → Actualizar

Lote 2 (32 muestras) → Calcular error medio → Actualizar

...

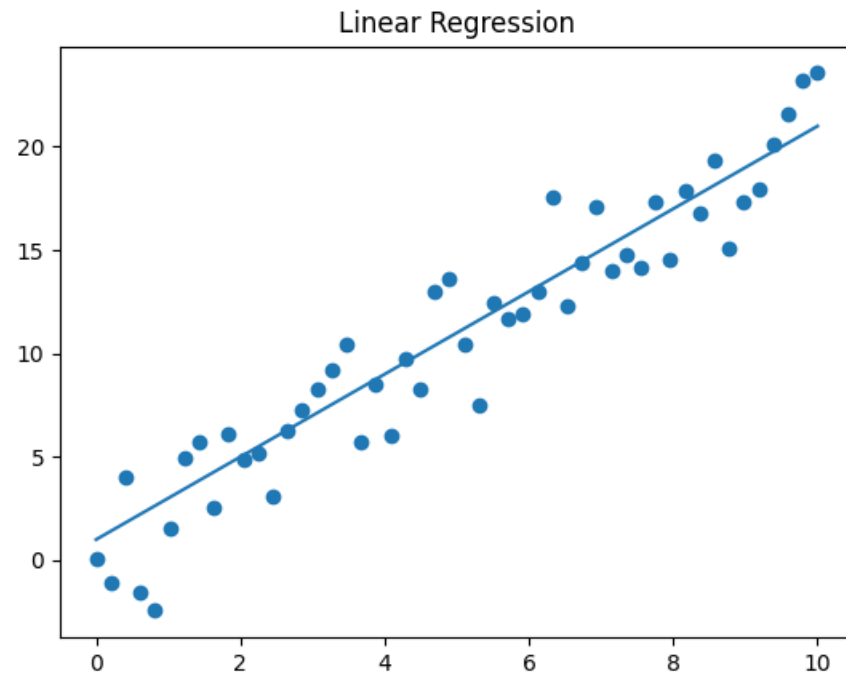
→ Repetir

La salida y calculada, se denota por $\hat{y}$

Conclusión en el **proceso** de regresión lineal:

- Si el error se calcula sobre todo el conjunto de datos antes de ajustar los parámetros, se trata de Batch Gradient Descent.
- Si se calcula el error y se actualizan los parámetros por cada instancia, se trata de Stochastic Gradient Descent.
- En la práctica, la mayoría de los frameworks modernos (TensorFlow, PyTorch, Keras) usan mayormente Mini-Batch Gradient Descent, que combina lo mejor de ambos enfoques, que promedia el error de pequeños grupos de instancias y actualiza después de cada mini-lote.

Linear Regression



Regresión Lineal: Función de coste (J) Error Cuadrático Medio (ECM o MSE en inglés).

- El ECM ($J(\theta)$) cuantifica la precisión con la que una función de hipótesis predice los datos.
- Calcula la diferencia cuadrática media entre los valores predichos y los reales para todos los ejemplos de entrenamiento.
- Este error se minimiza mediante una función de optimización para encontrar los mejores parámetros (θ_0, θ_1) para la hipótesis.

Ecuaciones Regresión Lineal

1. Modelo Lineal

$$h\theta(x) = \theta_0 + \theta_1x_1 + \theta_2x_2 + \dots + \theta_nx_n$$

2. Forma Vectorial

$$h\theta(x) = \theta^T x$$

donde $x = (1, x_1, x_2, \dots, x_n)$ y $\theta = (\theta_0, \theta_1, \dots, \theta_n)$

3. Función de Costo (Error Cuadrático Medio)

$$J(\theta) = (1 / 2m) * \sum (h\theta(x(i)) - y(i))^2$$

4. Interpretación

El modelo predice un valor continuo como una combinación lineal de las características de entrada. Los parámetros θ se ajustan minimizando la función de costo $J(\theta)$.

Se calcula de la siguiente manera:

1. Inicialización de parámetros: El algoritmo de regresión lineal comienza inicializando los parámetros θ_0 y θ_1 (para regresión lineal univariable) de manera aleatoria, lo que genera una función hipótesis.
 2. Cálculo de la diferencia: Para cada ejemplo en el conjunto de datos de entrenamiento, se resta la etiqueta real (Y_i) de la predicción de la función hipótesis ($H\theta(X_i)$).
 3. Elevación al cuadrado: El resultado de la resta se eleva al cuadrado para asegurar que el error sea siempre un valor positivo y para penalizar más los errores grandes.
 4. Suma de errores: Se suman todos los errores cuadrados de cada ejemplo en el conjunto de datos.
 5. Promedio: La suma total de los errores cuadrados se divide por el número total de ejemplos (m), y generalmente también por 2 ($2m$) porque en la práctica funciona un poco mejor.
- La ecuación de la función de coste ($J(\theta)$) es:

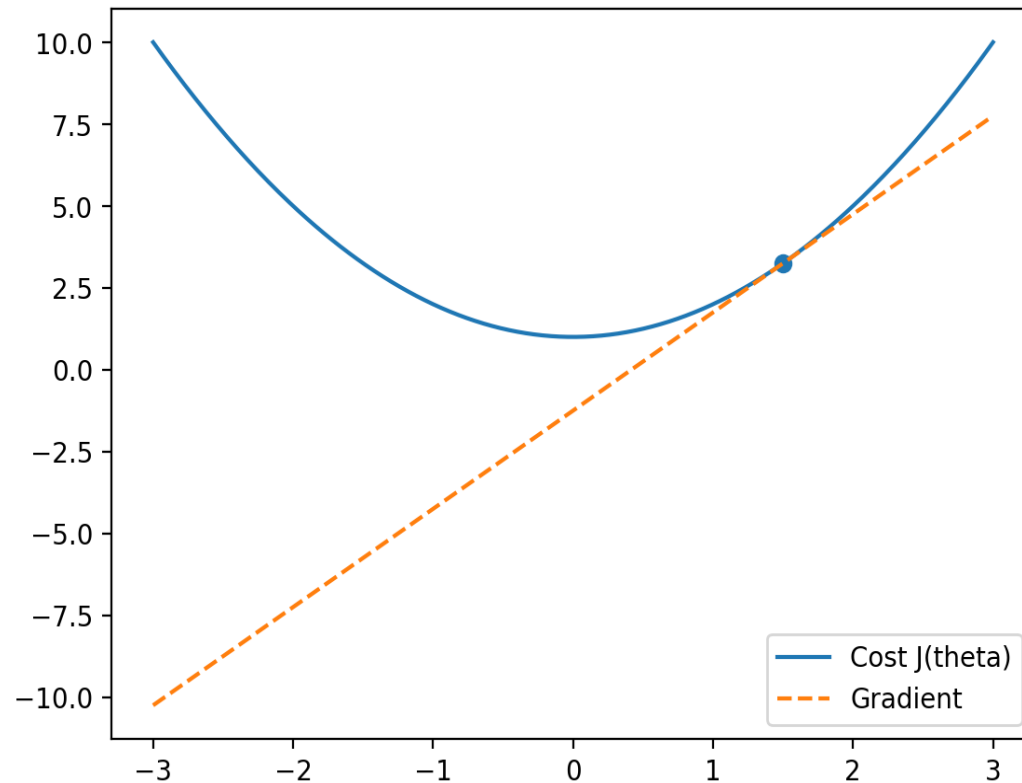
$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x_i) - y_i)^2$$

Regresión Lineal: Función de optimización (Descenso de gradiente)

1. Inicialización aleatoria: Los parámetros se inicializan con un valor aleatorio.
2. Cálculo de la derivada: Se calcula la derivada parcial de la función de coste con respecto a los parámetros en el punto actual. La derivada indica la pendiente de la función en ese punto.
3. Actualización de los parámetros: Los parámetros se actualizan restando el producto de una constante llamada "learning rate" y la derivada de la función de coste. La fórmula general para la actualización de un parámetro (θ) es:
$$\theta = \theta - (\text{learning rate} \times dJ(\theta)/d\theta)$$
4. Iteración: Este proceso se repite hasta que el valor de los parámetros no varíe significativamente, lo que indica que se ha alcanzado el punto mínimo de la función de coste.

El "learning rate" es una constante que se le debe indicar de antemano al algoritmo de gradient descent. Generalmente, su valor suele ser 0.5 (un medio), aunque puede modificarse. Su rol es indicar la velocidad con la que el algoritmo se acerca al punto óptimo.

Gradient Descent



El descenso por gradiente es un algoritmo de optimización que ajusta los parámetros del modelo. En cada iteración se actualizan los parámetros en la dirección que reduce el error.

Ejercicio Regresión Lineal

- Practicar cuaderno:

4_Regresión Lineal - Predicción del coste de un incidente de seguridad_CA.ipynb

Sección 7: Regresión Logística y Clasificación

Regresión Logística: Introducción

- La regresión logística predice valores discretos, a diferencia de la regresión lineal, que predice valores continuos.
- Utiliza un modelo lineal generalizado y aplica una función logística a la suma ponderada de las características de entrada y un sesgo.
- La principal aplicación que se analiza es la clasificación binaria, como la detección de spam, donde la salida es 0 o 1.

Regresión Logística

1. Modelo Lineal

$$z = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$

2. Forma Vectorial

$$z = \theta^T x$$

donde $x = (1, x_1, x_2, \dots, x_n)$ y $\theta = (\theta_0, \theta_1, \dots, \theta_n)$

3. Modelo de Regresión Logística

$$P(y=1 | x) = \sigma(\theta^T x)$$

Interpretación $P(y=1 | x) = \sigma(\theta^T x)$

- $P(y=1 | x)$ Es la **probabilidad** de que la salida sea 1 (clase positiva), dado un vector de entrada x . Ejemplo:

$y=1$: “el paciente tiene la enfermedad”

$y=0$: “no la tiene”

$\theta^T x$ Es un **modelo lineal**, igual que en regresión lineal:

$$\theta^T x = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$

Esto genera un valor que puede ser:

- negativo
- positivo
- cualquier número real
- **Problema**

Ese valor **no es una probabilidad** (puede ser mayor que 1 o menor que 0).

Solución: La Función Sigmoid

$$\sigma(z) = 1 / (1 + e^{-z}) = H(x) \text{ Función Hipótesis para la Regresión Logística}$$

Esta función transforma el valor z en una probabilidad entre 0 y 1.

Interpretación $P(y=1 | x) = \sigma(\theta^T x)$

Intuición importante

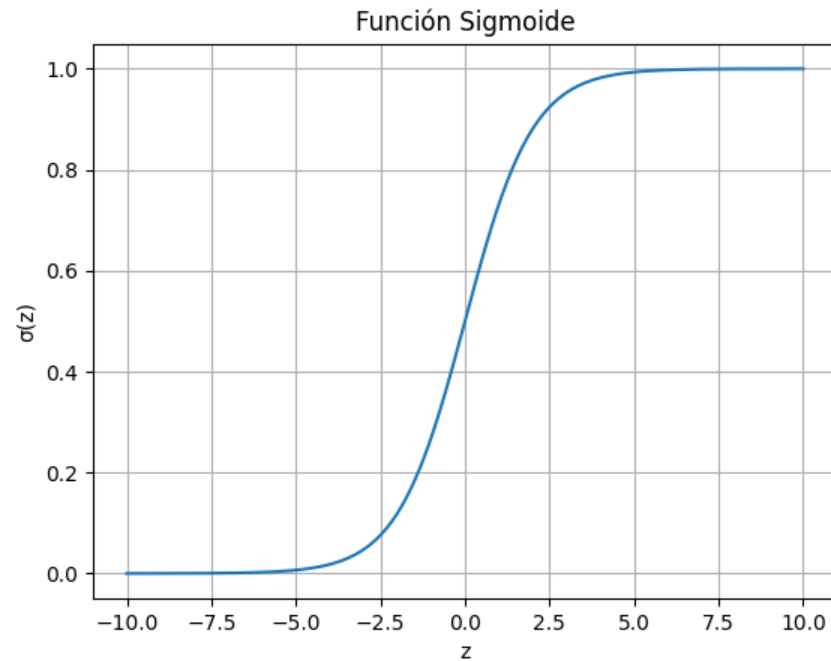
- Si $z \gg 0 \rightarrow \sigma(z) \approx 1 \rightarrow$ alta probabilidad de clase 1
- Si $z \ll 0 \rightarrow \sigma(z) \approx 0 \rightarrow$ baja probabilidad
- Si $z=0 \rightarrow \sigma(z)=0.5$

Es decir:

- La sigmoide convierte una **recta** en una **curva en forma de S**

En resumen: “La regresión logística toma una combinación lineal de las variables y la transforma en una probabilidad usando una función sigmoide.”

Regresión Logística: Función Sigmoid



La función sigmoide transforma valores en probabilidades entre 0 y 1. Esto permite interpretar la salida del modelo como la probabilidad de pertenecer a una clase.

Donde.

$$z = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$

$$\sigma(z) = 1 / (1 + e^{-z}) \text{ Función sigmoide}$$

Interpretación:

- Cuando $z = 0 \rightarrow \sigma(z) = 0.5$
- Cuando $z > 0 \rightarrow \sigma(z)$ se acerca a 1
- Cuando $z < 0 \rightarrow \sigma(z)$ se acerca a 0

Ejemplo práctico: probabilidad de aprobar un examen

Supongamos que queremos estimar la probabilidad de aprobar un examen en función de las horas de estudio (x).

Modelo logístico: $p = 1 / (1 + e^{-z})$

Asumimos $\theta_0 = -2$ y $\theta_1 = 0.8$.

Caso 1: $x = 5$ horas $\rightarrow z = -2 + 0.8(5) = 2 \rightarrow p \approx 0.88$

Caso 2: $x = 2$ horas $\rightarrow z = -2 + 0.8(2) = -0.4 \rightarrow p \approx 0.40$

- **Referencias**
- Chollet, F. (2021). Deep Learning with Python (2nd ed.). Manning Publications.
- Hastie, T., Tibshirani, R., & Friedman, J. (2017). The Elements of Statistical Learning (2nd ed.). Springer.
- Scikit-learn Documentation: Generalized Linear Models.
- Wikipedia contributors. Generalized linear model.

Regresión Logística: Representación de la función hipótesis

Limitaciones del uso de la función de hipótesis de la regresión lineal para problemas de clasificación.

- La función de hipótesis de la regresión lineal produce valores continuos, lo que dificulta la interpretación directa para la clasificación binaria (spam/no spam).
- Introducir un umbral (p. ej., 0.5) ayuda, pero los valores atípicos (outliers) pueden distorsionar la función lineal y provocar clasificaciones erróneas.
- La función también puede predecir valores fuera del rango deseado de 0 a 1, lo que requiere una nueva función de hipótesis para la regresión logística.

Regresión Logística: Función de coste

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log \sigma(\theta^T x^{(i)}) + (1 - y^{(i)}) \log(1 - \sigma(\theta^T x^{(i)}))]$$

¿Qué pasa con la ecuación de J cuando $y = 0$ o $y = 1$?

El objetivo es minimizar la función de coste J para encontrar los parámetros óptimos.

En resumen, la función de coste para la regresión logística tiene dos partes, una para cuando la etiqueta real es 1 y otra para cuando la etiqueta real es 0. Ambas partes están diseñadas para que el coste sea alto cuando el algoritmo realiza una mala predicción y bajo cuando realiza una buena predicción.

La no linealidad de la sigmoide está en la función de activación, pero lo que determina la existencia o no de mínimos locales es la convexidad de la función de costo.

Como la función de costo es convexa, la regresión logística NO tiene múltiples mínimos locales. En una función convexa, todos los mínimos locales coinciden con el mínimo global.

Regresión Logística: Algoritmo de optimización

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log \sigma(\theta^T x^{(i)}) + (1 - y^{(i)}) \log(1 - \sigma(\theta^T x^{(i)}))]$$

- **Regla general de actualización:** $\theta_j \leftarrow \theta_j - \alpha \frac{\partial J(\theta)}{\partial \theta_j}$

- **Gradiente de la función de costo:**

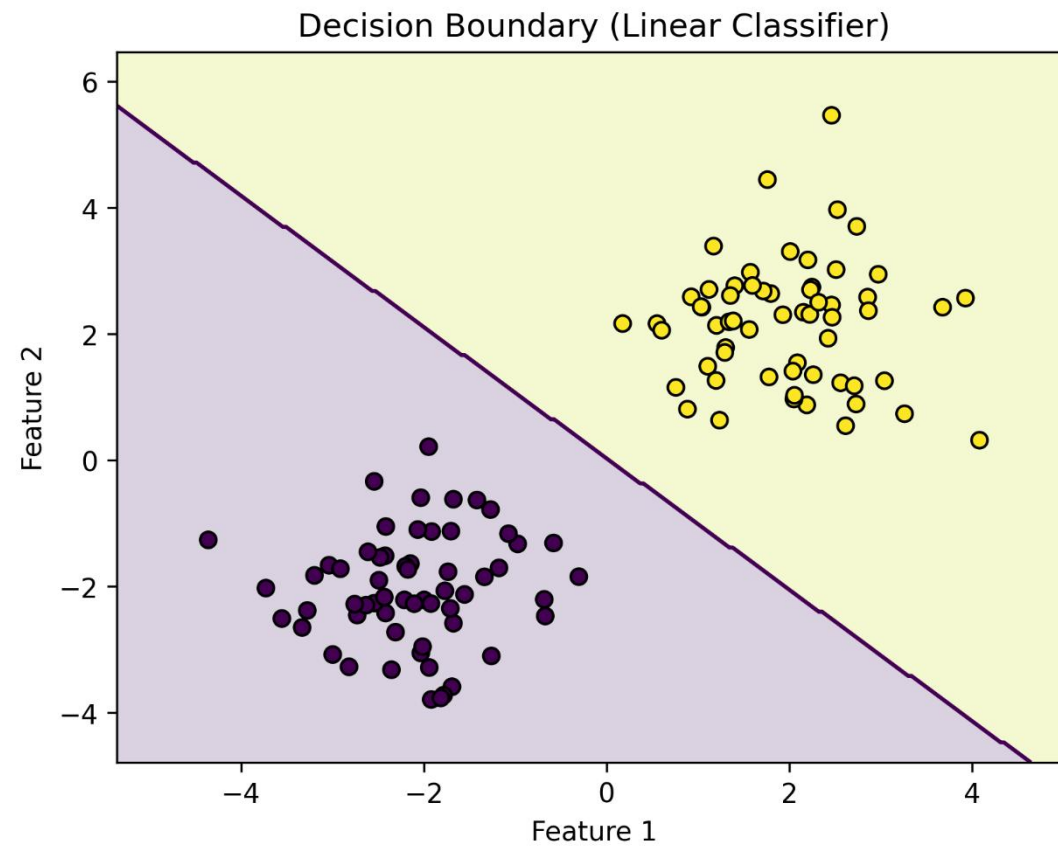
$$\frac{\partial J(\theta)}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

- **Regla de actualización completa:**

$$\theta_j \leftarrow \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

- *Nota: α representa el learning rate o tasa de aprendizaje. Se denominará un hiperparámetro*

- La ecuación $\theta^T x = 0$ define una **frontera de decisión**
- Esta frontera es una:
 - **recta** (en 2D)
 - **plano** (en 3D)
 - **hiperplano** (en dimensiones mayores)
- Esa línea es la que separa las clases en la siguiente gráfica.
 - Un lado de la recta es para: $\theta^T x > 0$, $\sigma(z) > 0.5$, o sea Clase 1
 - El otro lado de la recta es para: $\theta^T x < 0$, $\sigma(z) < 0.5$, o sea Clase 0
- La sigmoide define la probabilidad, pero la recta define la decisión



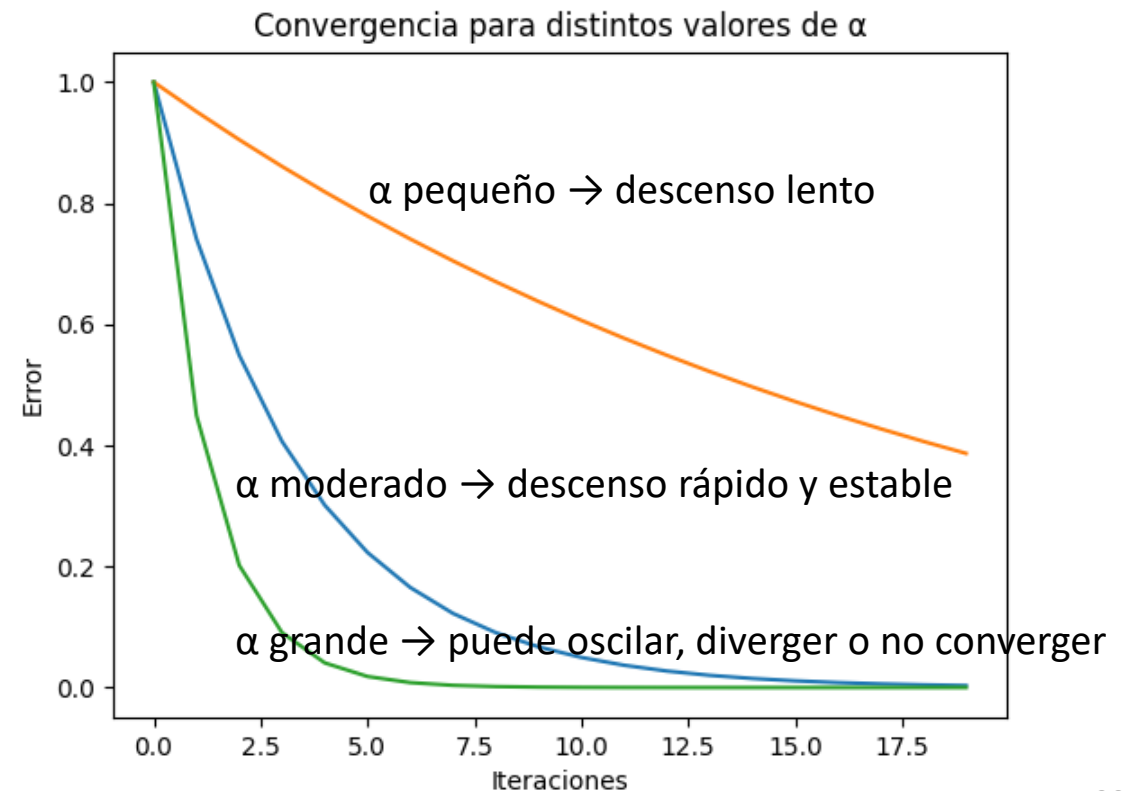
Concepto de Hiperparámetro en ML

“Los parámetros se aprenden; los hiperparámetros se eligen.”

$\theta \rightarrow$ qué aprende el modelo

$\alpha \rightarrow$ qué tan rápido aprende

Impacto de α en la convergencia



Ejercicio Regresión Logística

- Practicar cuaderno:

5_Regresión Logística - Detección de SPAM_CA.ipynb

1. Regresión Lineal

Modelo: $h\theta(x) = \theta^T x$

Se utiliza para predecir valores continuos.

2. Regresión Logística

Modelo: $h\theta(x) = \sigma(\theta^T x)$

Sigmoide: $\sigma(z) = 1 / (1 + e^{-z})$

3. Interpretación

El modelo sigue siendo lineal internamente, pero se transforma en probabilístico.

4. Problema del modelo lineal

- Valores fuera de $[0,1]$
- No interpreta probabilidades
- No sirve directamente para clasificación

5. Función de coste

$$J(\theta) = -(1/m) \sum [y \log(h\theta(x)) + (1-y) \log(1 - h\theta(x))]$$

Elemento	Regresión Lineal	Regresión Logística
Modelo	$\theta^T x$	$\sigma(\theta^T x)$
Salida	Continua	Probabilidad
Coste	MSE	Log-loss
Convexidad	Sí	Sí

- La regresión logística es un modelo lineal al que se le aplica una función no lineal para convertirlo en un modelo de probabilidades.
- $h_{\theta}(x) = \sigma(\theta^T x)$ **CORREGIR** donde la sigmoide es:
- $\sigma(z) = \frac{1}{1 + e^{-z}}$ **CORREGIR**
- ¿Por qué no basta con el modelo lineal?

Porque:

- Puede dar valores fuera de $[0, 1]$
- No representa probabilidades
- No es adecuado para clasificación
- No es solo cambiar la función de coste.
La regresión logística nace porque el modelo lineal no sirve para clasificar.
Por eso se transforma el modelo y luego se ajusta la función de coste para evitar los óptimos locales, o sea, que sea convexa.
- Ver “Evitar óptimos locales” en ChatGPT

- Pero eso no fue suficiente...
- También se modifica la función de coste
- Porque:
- MSE + sigmoide → **X** no convexa (tiene varios óptimos locales)
- Se reemplaza por **log-loss**:
- $J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y(i) \log(h_{\theta}(x(i))) + (1 - y(i)) \log(1 - h_{\theta}(x(i)))]$ **CORREGIR**

Sección 8: Etapas de un proyecto de ML

Consideraciones

- Si bien algoritmos como la regresión lineal y logística son fundamentales, la mayor parte del tiempo en proyectos reales se dedica al procesamiento y la transformación de datos.
- La calidad y la cantidad de datos son tan importantes como el propio algoritmo para un aprendizaje automático eficaz.
- Si se tiene un conjunto de datos lo suficientemente grande y representativo del problema, la selección del algoritmo se convierte en un problema casi secundario, ya que muchos algoritmos se comportarán con una precisión y rendimiento similar al predecir nuevos ejemplos.

Recomendaciones claves para recopilar y preparar un conjunto de datos de entrenamiento

- **Disponer de una cantidad suficiente de datos:** Cuantos más datos se tengan, mejor se comportarán los algoritmos de machine learning. Con una mayor cantidad de datos, la selección del algoritmo se vuelve menos crítica, ya que la mayoría de los algoritmos alcanzarán un rendimiento y precisión similares, y este rendimiento será mayor cuanto más datos se utilicen.
- **Utilizar un conjunto de datos representativo del problema práctico:** Es esencial que las características de entrada o atributos extraídos sean representativas para diferenciar, por ejemplo, una transacción fraudulenta de una legítima. Extraer atributos que no tienen relación con el problema o que no marcan una diferencia significativa entre los tipos de ejemplos puede llevar a un peor rendimiento del algoritmo.
- **Los datos deben ser de calidad:** El conjunto de datos debe consistir en datos reales y no en datos generados manualmente o de forma artesanal. Utilizar datos no reales o de baja calidad impedirá que el algoritmo prediga correctamente para situaciones reales.
- **Seleccionar un conjunto de características adecuado, minimizando el número de características irrelevantes:** Cuantas más características de entrada se tengan, más recursos computacionales necesitará el algoritmo y más complejo será el modelo. Si se incluyen muchas características irrelevantes, el rendimiento del algoritmo puede deteriorarse y los recursos computacionales necesarios para el entrenamiento aumentarán considerablemente. Es importante seleccionar características que sean significativas para el problema.

Caso práctico: Visualización del conjunto de datos

- Ver el cuaderno 6_Visualización del conjunto de datos_CA.ipynb
- El conjunto de datos utilizado en este caso práctico es NSLKDD, que clasifica el tráfico de red como normal o anómalo.
- El conjunto de datos NSLKDD se deriva de tráfico de red real, donde cada fila representa un flujo de red.
- En el notebook se abordan métodos para leer datos en formatos .ARFF y .txt (CSV) utilizando bibliotecas de Python como Pandas.
- Cubre técnicas básicas de visualización de datos, incluyendo la impresión de las primeras filas, ``info()``, ``describe()``, ``value_counts()``, histogramas y análisis de correlación para comprender la estructura y las relaciones de los datos.

Visualización de datos

- **Inspección de las primeras filas:** Se pueden imprimir las primeras 10 filas del conjunto de datos para ver las características de entrada extraídas para cada flujo de tráfico de red y la característica de salida (la clase).
- **Información básica del conjunto de datos:** El método `.info()` proporciona información básica sobre cada característica de entrada, indicando cuáles son numéricas (float) y cuáles son categóricas (Object). Por ejemplo, `protocol_type` se identifica como una variable categórica.
- **Estadísticas descriptivas:** Funciones más avanzadas pueden proporcionar información estadística sobre las características numéricas, como la media, la desviación estándar, los percentiles, el valor mínimo y el valor máximo. Esto ayuda a comprender la formación de estas características y sus rangos.
- **Valores únicos y conteo:** Para atributos categóricos como `protocol_type` o la `class`, se puede usar `value_counts()` para ver las categorías únicas y cuántos ejemplos (flujos) tienen cada valor. Por ejemplo, para `protocol_type`, se puede ver que los valores únicos son TCP, UDP e ICMP, y el número de ejemplos para cada uno. Para la clase, se puede ver cuántos flujos están etiquetados como "normal" y cuántos como "anómalo".
- **Histogramas:** Se pueden representar los valores de una característica en un histograma para visualizar su distribución. Esto permite ver las proporciones de cada valor que recibe una variable o característica de entrada, como el `protocol_type`. También se pueden representar histogramas para todos los atributos de entrada para ver su distribución.

Análisis de correlación

- **Coeficiente de correlación estándar:** Se puede calcular el coeficiente de correlación estándar para ver la correlación entre cada par de atributos. Este coeficiente mide correlaciones lineales, indicando si el valor de un atributo crece o disminuye en proporción al crecimiento o disminución de otro.
- **Correlación entre atributos de entrada y la variable de salida:** Es útil medir la correlación entre los atributos de entrada (X_1, X_2, X_n) y la variable de salida (Y). Una alta correlación indica que ese atributo de entrada es significativo para clasificar los ejemplos en una categoría u otra. Para esto, se pueden transformar los valores de la clase de categóricos a numéricos (por ejemplo, "normal" a 1 y "anómalo" a 0) para poder calcular la correlación.
- **Correlación entre atributos de entrada:** Si dos características de entrada están muy correlacionadas, es posible que estén proporcionando la misma información. En este caso, se podría eliminar una de ellas para simplificar el conjunto de datos, reducir la complejidad y los recursos computacionales necesarios para entrenar el algoritmo de machine learning.
- **Matriz de correlación:** Se puede mostrar la correlación entre todos los atributos utilizando el método `.corr()`. Esto genera una matriz numérica que muestra la correlación entre cada par de atributos.
- **Representación gráfica de la matriz de correlación:** La matriz de correlación se puede representar gráficamente. Cuanto más amarillo sea un punto en la representación gráfica, mayor será la correlación. Esto puede ayudar a identificar visualmente atributos altamente correlacionados.
- **Scatter matrix:** Para un subconjunto de atributos, se puede utilizar un scatter matrix para representar gráficamente la correlación y la dirección de esta correlación entre ellos.

En resumen, la visualización de datos permite ...

- comprender la composición y distribución de los flujos de tráfico de red, mientras que el análisis de correlación ayuda a identificar relaciones entre las características de entrada y la clase de salida, así como entre las propias características de entrada, lo que es crucial para la construcción de modelos de aprendizaje automático.

Procesamiento de conjuntos de datos en diferentes formatos como .txt (CSV) y .ARFF

Pasos y Desafíos al leer un archivo .txt en formato CSV

1. Lectura básica con Python:

- **Paso:** Se puede utilizar la función ``open`` de Python para leer el archivo línea por línea.
- **Desafío:** El resultado es una lista de cadenas de texto, donde cada cadena representa una línea del archivo. Estas cadenas contienen los valores separados por comas y un salto de línea (``\\n``). Esto es una representación "bastante fea" y "poco útil" ya que requiere "muchas tareas de preprocesamiento" como recorrer la lista, parsear las cadenas de texto, eliminar el ``\\n`` y separar por comas.

2. Lectura con Pandas:

- **Paso:** Se utiliza la función ``read_csv`` de la librería Pandas.
- **Desafío:** Si el archivo CSV no incluye los nombres de las características de entrada en la primera fila, Pandas puede interpretar erróneamente la primera fila de datos como los nombres de las columnas. Esto significa que los nombres de las características (X_1, X_2, X_N) no se cargan correctamente, y la primera fila de datos se pierde o se usa incorrectamente como encabezado.

Pasos y Desafíos al leer un archivo .ARFF

1. Identificación del formato:

- * **Paso:** Se identifica que el archivo tiene la extensión .ARFF, que es un formato de texto ASCII que describe una lista de instancias (un conjunto de datos) con atributos.

2. Ausencia de función directa en Pandas:

- * **Desafío:** Pandas no tiene una función preimplementada para leer archivos .ARFF directamente.

3. Instalación de una librería externa:

- * **Paso:** Es necesario instalar un paquete de Python adicional, como `liac-arff`, para poder leer este tipo de archivos.

4. Lectura y procesamiento con la librería externa:

- * **Paso:** Se utiliza la función `open` de Python para abrir el archivo .ARFF y luego la función `load` de la librería `liac-arff` para cargarlo e interpretar sus valores.

- * **Resultado:** La lectura con esta librería devuelve un diccionario.

- * **Desafío:** Este diccionario contiene claves como `data` (que representa los valores de las características de entrada) y `attributes` (que contiene los nombres de los atributos y su tipo, como si son numéricos o categóricos, y las categorías para los categóricos). Es necesario extraer y procesar estas claves para obtener los datos y los nombres de las columnas por separado.

Pasos y Desafíos al leer un archivo .ARFF (cont.)

5. Transformación a DataFrame de Pandas:

- * **Paso:** Una vez que se han extraído los datos y los nombres de los atributos del diccionario ARFF, se transforman en un DataFrame de Pandas. Esto se hace pasando los datos y la lista de nombres de atributos como columnas.

- * **Beneficio:** Pandas representa los datos de una manera más organizada y facilita la aplicación de transformaciones.

Práctica recomendada para ambos formatos:

- * **Creación de funciones de lectura:** Es una buena práctica construir una función que encapsule el proceso de lectura y procesamiento de un formato específico (por ejemplo, ARFF) para que el código sea más modular y reutilizable. Esta función debería leer el archivo, extraer los datos y los nombres de los atributos, y devolver un DataFrame de Pandas ya estructurado.

Entrenamiento del modelo de ML

- Para obtener un modelo adecuado (que generalice bien) se entrena de la siguiente manera:
- El conjunto de datos inicial se divide en 3 diferentes subconjuntos. Se utiliza un subconjunto para entrenar (train set) el algoritmo, otro para validar (validate set) que un modelo funciona correctamente y un tercero para probar (test set) que el algoritmo y el modelo generalizan bien para nuevos ejemplos.
- En realidad, primero se divide el conjunto de datos en conjunto de entrenamiento y conjunto de prueba (test). Posteriormente, el conjunto de entrenamiento se subdivide en un conjunto de entrenamiento propiamente tal y un conjunto para validar.
- **Importante:** No se debe abrir y observar o hacer cambios (manipulación) en el conjunto de datos de Test

¿Qué es Overfitting y Underfitting?

- **Problema:** En machine learning, el overfitting (sobreajuste) y el underfitting son problemas que surgen al aplicar técnicas de aprendizaje automático a problemas reales.
- El **overfitting** ocurre cuando la función hipótesis generada por el algoritmo se adapta muy bien al conjunto de datos de entrenamiento, minimizando el error hasta cero, pero falla al generalizar con nuevos ejemplos. Esto significa que el modelo se adapta perfectamente a los ejemplos del conjunto de datos de entrenamiento (experiencia pasada) (**memorización vs aprendizaje**), pero cuando se le presentan nuevos ejemplos, no se comporta bien y no es capaz de realizar buenas predicciones para datos que no ha visto nunca. Un modelo con muchas características polinómicas, que lo hacen muy flexible, puede causar overfitting al adaptarse demasiado a los datos pasados y generar un límite de decisión que no predice correctamente para ejemplos nuevos.
- El **underfitting** es el caso contrario al overfitting. Se produce cuando el modelo es demasiado rígido y no es apropiado para ajustarse a la tendencia del conjunto de datos. Por ejemplo, si la tendencia de los datos es curva y el modelo solo puede generar una línea recta, el error será grande. Esto significa que el algoritmo o la función hipótesis es muy rígida y no se puede adaptar bien al conjunto de datos, lo que resulta en un error significativo incluso para los datos de entrenamiento y, por lo tanto, en predicciones incorrectas para nuevos ejemplos.
- Es igualmente perjudicial producir underfitting como overfitting, ya que ambos impiden que el modelo generalice bien para nuevos ejemplos. Si bien el **underfitting** se resuelve aumentando la flexibilidad del modelo, el **overfitting** requiere estrategias más complejas.

Soluciones al Overfitting

1. **Aumentar el conjunto de datos:** Si se tiene la posibilidad de recopilar más datos de entrenamiento, un conjunto de datos más grande y representativo de los datos futuros reducirá el sobreentrenamiento. Esto se debe a que, con más datos, es más probable que un nuevo dato se parezca a alguno de los ejemplos ya existentes en el conjunto de entrenamiento, lo que permite una predicción más adecuada incluso si hay sobreentrenamiento.
2. **Reducción del número de características:** El sobreentrenamiento puede ocurrir cuando se utilizan demasiadas características, especialmente características polinómicas, que hacen que el modelo sea demasiado flexible. Reducir el número de características puede hacer el modelo más rígido y menos propenso al sobreentrenamiento. Esto se puede lograr de varias formas:
 - * **Selección manual de características:** Elegir manualmente qué características mantener, aunque no es muy recomendable para un gran número de características.
 - * **Algoritmos de selección de características:** Utilizar algoritmos (como Random Forest) que identifican las características importantes y permiten eliminar las irrelevantes.
 - * **Algoritmos de extracción de características** (reducción de dimensionalidad): Estos algoritmos (como PCA o SVD) transforman el conjunto de datos con N características en uno nuevo con menos características (por ejemplo, $N/2$), manteniendo la distribución original de los datos.

Soluciones al Overfitting (cont.)

3. Regularización: Esta es una de las técnicas más comunes y consiste en añadir una penalización a determinados parámetros del modelo para reducir su flexibilidad. La intuición es que valores más grandes de los parámetros (θ_0 , θ_1 , etc.) hacen el modelo más flexible. La regularización busca reducir estos valores para aquellos parámetros que causan sobreentrenamiento, disminuyendo la flexibilidad del modelo para que no se ajuste tan perfectamente al conjunto de datos de entrenamiento.
 - * La regularización mantiene todas las características de entrada, pero reduce la magnitud de los parámetros del modelo.
 - * Añade un término de penalización a la función de coste, determinado por un hiperparámetro λ .
 - * Un valor de λ igual a cero significa que no hay regularización.
 - * Un valor de λ muy grande (cercano a infinito) puede llevar a que los parámetros (excepto θ_0) se minimicen a cero, haciendo que el modelo no realice buenas clasificaciones.
 - * El objetivo es encontrar un valor de λ que penalice el modelo lo suficiente para que generalice mejor a ejemplos nuevos, sin que se ajuste demasiado a los datos de entrenamiento.
 - * Ejemplos de regularización incluyen Ridge Regression y Lasso.
 - * Funciona muy bien cuando se tienen muchas características de entrada que son ligeramente útiles.

Regularización

1. Idea central

La regularización permite controlar la complejidad del modelo, evitando el sobreajuste y mejorando la generalización.

2. Función de costo

Sin regularización:

$$J(\theta) = (1 / 2m) \cdot \sum (h_{\theta}(x) - y)^2$$

Con regularización L2 (Ridge):

$$J(\theta) = (1 / 2m) \cdot \sum (h_{\theta}(x) - y)^2 + (\lambda / 2m) \cdot \sum \theta^2$$

Penaliza pesos grandes. Obliga a que los θ sean pequeños

Resultado: La función es más “suave” y menos sensible al ruido

Con regularización L1 (Lasso):

$$J(\theta) = (1 / 2m) \cdot \sum (h_{\theta}(x) - y)^2 + (\lambda / 2m) \cdot \sum |\theta|$$

3. Interpretación del hiperparámetro λ

λ controla el equilibrio entre ajuste y simplicidad del modelo:

Error total = Error de entrenamiento + λ · Complejidad del modelo

λ bajo → modelo complejo (riesgo de overfitting)

λ alto → modelo simple (riesgo de underfitting)

Regularización L2 (Ridge)

- ¿Qué significa “menos sensible al ruido” en L2?: El modelo **no reacciona exageradamente** a pequeñas variaciones o errores en los datos.
- Pero, **¿qué es el ruido?** En un dataset, el ruido puede ser:
 - ❖ errores de medición
 - ❖ valores atípicos (outliers)
 - ❖ variaciones aleatorias
- Ejemplos:
 - Precio de casa mal registrado
 - Sensor que mide mal
 - dato “raro” que no representa el patrón real
- ¿Y por qué eso reduce la sensibilidad al ruido?

Pesos grandes = modelo “nervioso”

Pesos pequeños = modelo “suave”

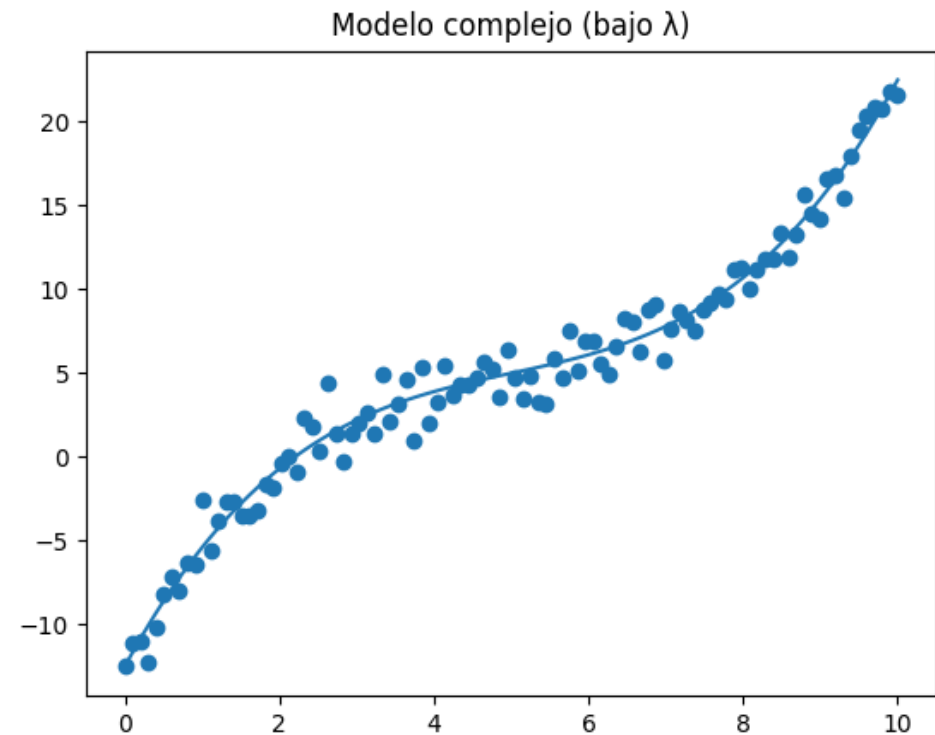
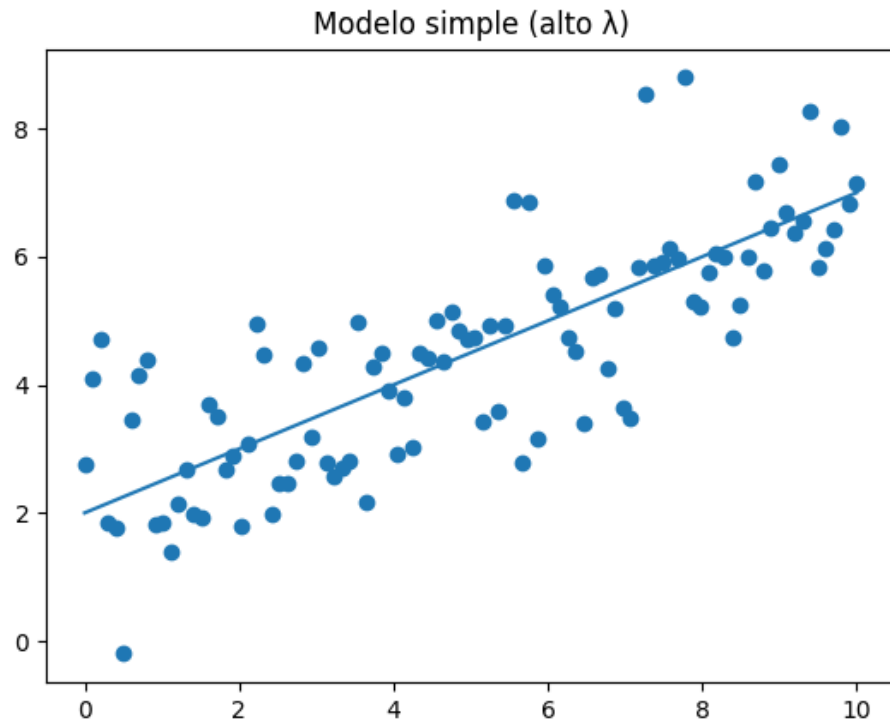
Si los pesos son grandes:

- pequeñas variaciones en $x \rightarrow$ generan grandes cambios en la salida $h(x)$

Si los pesos son pequeños:

- pequeñas variaciones en $x \rightarrow$ generan cambios pequeños en la salida. El modelo se vuelve **estable**

Visualización conceptual



La regularización no busca mejorar el ajuste en entrenamiento, sino mejorar el desempeño en datos nuevos.

L1 vs L2

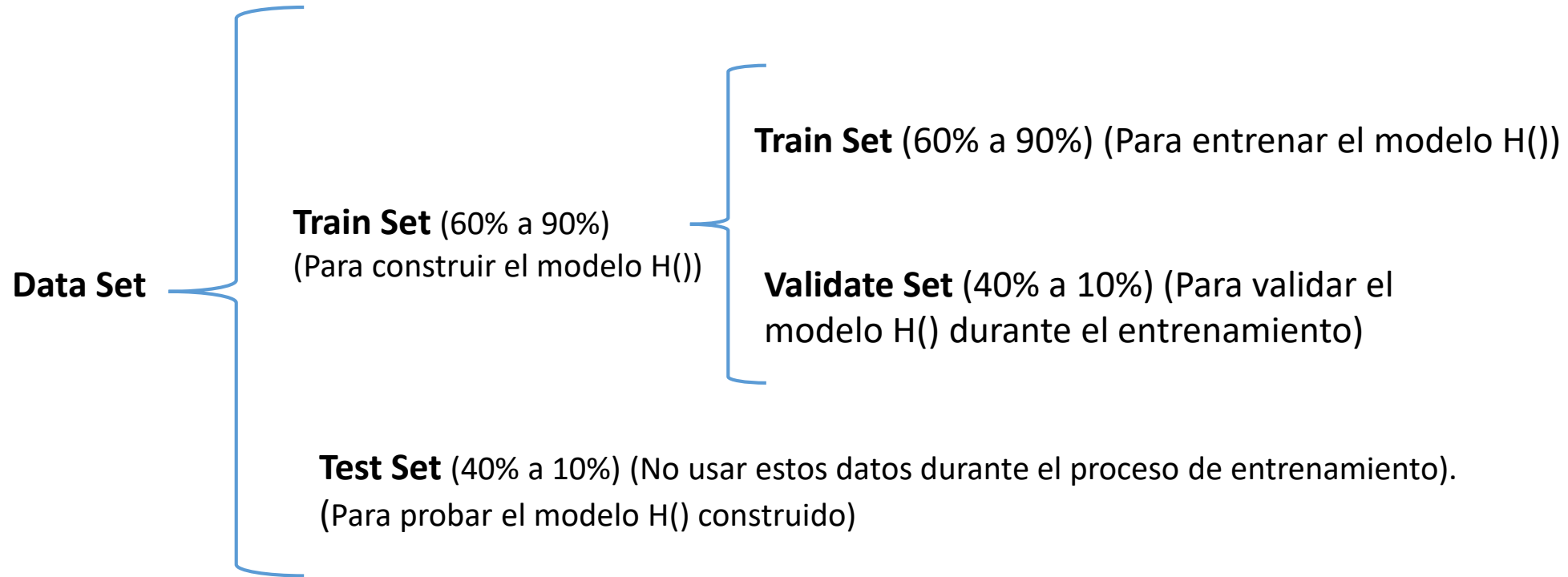
Similitudes entre L1 y L2

- Evitan el sobreajuste al penalizar la complejidad del modelo.
- Modifican la función de costo añadiendo un término de regularización.
- Actúan sobre los parámetros del modelo (θ), reduciendo su magnitud.
- Introducen un hiperparámetro λ que controla la intensidad de la penalización.

Evaluación de la función hipótesis (H)

- Se entrena el algoritmo y se construye la función hipótesis utilizando únicamente el subconjunto de entrenamiento (60% a 90% del Conjunto de Datos). **IMPORTANTE:** El subconjunto de entrenamiento se subdivide en entrenamiento (60% a 90% y validación (40% a 10%)
- Para comprobar si la función hipótesis está produciendo overfitting, se utilizan las características de entrada (X_1) del subconjunto de pruebas (40% a 10% del Conjunto de Datos). Estos ejemplos del subconjunto de pruebas no han sido vistos por el algoritmo durante la construcción de la función hipótesis.
- Se realizan predicciones con la función hipótesis utilizando las características de entrada del subconjunto de pruebas.
- Finalmente, se comparan estas predicciones con las etiquetas reales (el valor real) que tienen los ejemplos en el subconjunto de pruebas.

Importante de entender



Desarrollo del modelo

- En esta etapa, el modelo H se entrena utilizando el **subconjunto de entrenamiento (Train Set)**, ajustando sus parámetros para minimizar el error entre sus predicciones y las etiquetas reales de dicho conjunto.
- Luego, se utiliza el **subconjunto de validación (Validation Set)** para evaluar el desempeño del modelo con datos que no han sido vistos durante el entrenamiento. Esto permite ajustar hiperparámetros y seleccionar el modelo que mejor generaliza.
- Para detectar posibles problemas de overfitting, se comparan los resultados obtenidos en el Train Set y en el Validation Set. Si el modelo tiene un desempeño significativamente mejor en entrenamiento que en validación, es probable que esté sobreajustado.
- Finalmente, una vez seleccionado el modelo adecuado, se evalúa utilizando el **subconjunto de prueba (Test Set)**, el cual no ha sido utilizado ni en el entrenamiento ni en la validación. Esta evaluación proporciona una estimación objetiva del rendimiento del modelo en datos nuevos.

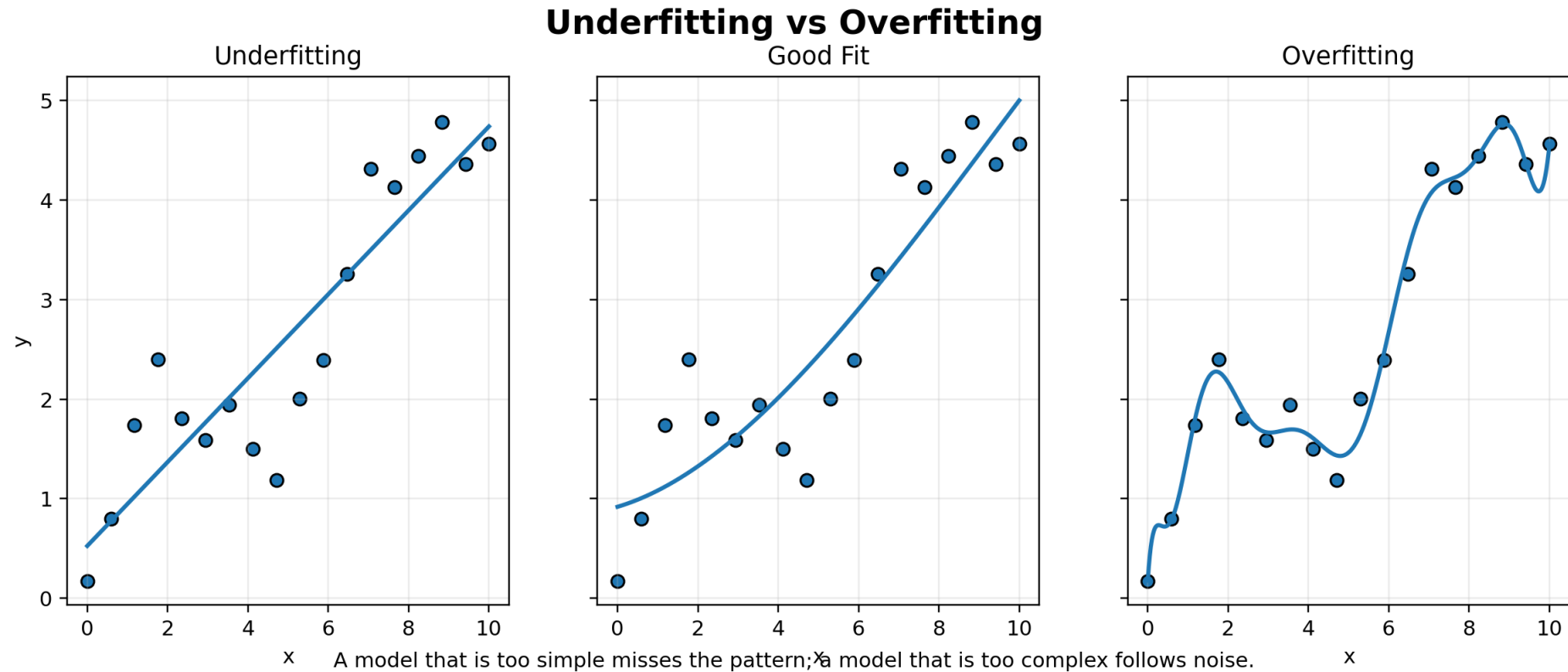
Desarrollo del modelo

- **Train Set** → aprender
- **Validation Set** → elegir el mejor modelo
- **Test Set** → evaluar de forma final
- Si la accuracy de **entrenamiento** es mucho mayor que la de **validación**, hay overfitting.

Detección del Overfitting

- Si el error del modelo sobre el **subconjunto de entrenamiento (Train Set)** es muy bajo (cercano a cero), pero el error sobre el **subconjunto de validación (Validation Set)** o el **subconjunto de prueba (Test Set)** es significativamente mayor, entonces se está produciendo **overfitting**.
- Esto indica que el modelo se ha ajustado excesivamente a los datos de entrenamiento, capturando incluso el ruido presente en ellos, lo que impide que generalice correctamente cuando se enfrenta a ejemplos nuevos.
- La partición del conjunto de datos en subconjuntos de **entrenamiento, validación y prueba** se justifica precisamente para detectar este tipo de problemas. Mientras el subconjunto de entrenamiento se utiliza para ajustar el modelo, el subconjunto de validación permite evaluar su capacidad de generalización durante el proceso de selección del modelo, y el subconjunto de prueba se utiliza como una evaluación final e independiente.
- Esta metodología es especialmente importante en problemas con un elevado número de características de entrada, donde no es posible realizar una inspección visual del comportamiento del modelo.

Overfitting vs Underfitting



El underfitting ocurre cuando el modelo es demasiado simple para capturar patrones.
El overfitting ocurre cuando el modelo se ajusta demasiado a los datos de entrenamiento.

Selección del modelo

- El conjunto de entrenamiento se usa para entrenar los modelos y el conjunto de validación se usa para evaluar si se está produciendo sobreajuste sobre el conjunto de entrenamiento y así, seleccionar el modelo óptimo.
- Posteriormente, el conjunto de prueba se utiliza, en última instancia, para verificar que el modelo seleccionado generaliza bien a ejemplos que no ha visto anteriormente y que no están ni en el conjunto de validación ni en el de entrenamiento.

Caso Práctico: División del conjunto de datos

- Usar el notebook: 7_División del conjunto de datos_CA.ipynb

Objetivo

- Dividir un conjunto de datos en subconjuntos de entrenamiento, validación y prueba.
- Se divide primero el conjunto de entrenamiento en subconjunto de entrenamiento y conjunto de prueba (test), posteriormente se toma el subconjunto de entrenamiento y se divide en subconjunto de entrenamiento y subconjunto de validación.
- El proceso implica usar `train\_test\_split` de Scikit-learn, para crear los conjuntos iniciales de entrenamiento y prueba.
- **Shuffle**: Mezcla los datos para evitar sesgos por orden
- **Stratify**: Asegura que cada subconjunto tenga la misma proporción de clases
- Ambos no son hiperparámetros. Son **parámetros** del proceso de preparación de datos.
- Un hiperparámetro es un valor que controla **cómo se entrena el modelo**

Caso Práctico: Preparación del conjunto de datos

- Ver el cuaderno 8_Preparación del conjunto de datos_CA.ipynb
- **Tratamiento de valores faltantes:** Las estrategias incluyen eliminar el ejemplo, eliminar la característica o imputar un valor (como la media).
- **Datos categóricos:** Técnicas como la codificación one-hot encoding y la codificación ficticia transforman las categorías no numéricas en representaciones numéricas.
- **Escalado de características:** El escalado de media-máxima (normalización a 0-1) y la estandarización (media 0, desviación estándar 1) abordan las disparidades en las escalas numéricas.
- **Desequilibrio de datos:** Métodos como el sobremuestreo (duplicación de la clase minoritaria) y el submuestreo (reducción de la clase mayoritaria) abordan los conjuntos de datos sesgados.

Tratamiento de valores faltantes

Es común que los datos recopilados de problemas reales contengan valores perdidos para ciertas características. Los algoritmos de machine learning no pueden procesar conjuntos de datos con valores nulos o vacíos. Para abordar esto, existen tres formas comunes:

- **Eliminar el ejemplo:** Si un ejemplo (fila) tiene un valor perdido para una característica, se puede eliminar esa transacción del conjunto de datos.
- **Eliminar la característica:** Si una característica (columna) tiene valores perdidos en la mayoría de los ejemplos, especialmente si es difícil de recopilar, se puede optar por eliminar esa característica por completo.
- **Añadir un valor determinado:** Se puede reemplazar el valor perdido con un valor calculado, como la media aritmética o la mediana de los valores existentes para esa característica.

Datos categóricos

La mayoría de los algoritmos de ML requieren valores numéricos para funcionar. Sin embargo, los conjuntos de datos reales a menudo incluyen características categóricas (por ejemplo, métodos de pago como PayPal, tarjeta de crédito). Para transformar estas variables categóricas en numéricas, se utilizan técnicas de codificación:

- **One Hot Encoding:** Utiliza un array de bits, con tantos bits como valores diferentes tenga la característica categórica. Se activa un bit (se pone en positivo) por cada categoría.
- **Dummy Coding (o Dummy Encoding):** Reduce el grado de libertad utilizando $K-1$ nuevas características, donde K es el número de posibles valores distintos para la característica categórica.

Datos categóricos: Ejemplos

1. One Hot Encoding:

- Utiliza un array de bits, con tantos bits como valores diferentes tenga la característica categórica.
- Activa (pone como positivo) un bit por cada una de esas categorías.
- Por ejemplo, si un método de pago (característica X4) puede ser PayPal, Credit Card, Bizum u otro tipo de pago (cuatro tipos diferentes), se selecciona un array de 4 bits.
 - * PayPal se sustituiría por [1 0 0 0] (activando el primer bit).
 - * Credit Card activaría el segundo bit. [0 1 0 0]
 - * Bizum activaría el tercer bit [0 0 1 0]
- De esta manera, los valores categóricos se reemplazan por un vector numérico con un bit activado dependiendo de la categoría.

2. Dummy Coding (o Dummy Encoding):

- Reduce el grado de libertad utilizando K-1 nuevas características, donde K es el número de posibles valores distintos para la característica categórica.
- Por ejemplo, si el método de pago tiene tres valores diferentes (PayPal, Credit Card y Bizum):
 - * PayPal se denotaría con un vector de dos bits, como `1 0`.
 - * Credit Card se denotaría con `0 1`.
 - * Cualquier otra categoría (en este caso, Bizum) se representaría con el vector `0 0`.
- * Esta técnica reduce el número de bits necesarios en comparación con One Hot Encoding.

Escalado de características (Valores dispares o diferentes escalas)

Las características de entrada pueden tener valores numéricos con escalas muy diferentes (por ejemplo, un valor de 28 y otro de 0.002). Algunos algoritmos de machine learning no funcionan correctamente cuando los valores de los atributos están escalados de manera diferente. Es necesario homogeneizar los valores para que se mantengan dentro de un rango similar, conservando la distribución de los datos originales. Para esto, se utilizan dos mecanismos principales:

- **Min Max Scaling (Normalización):** Escala los valores de las características numéricas a un rango entre 0 y 1. Se calcula restando el valor mínimo de la característica y dividiendo por la diferencia entre el máximo y el mínimo.
- **Estandarización:** Cambia la distribución de cada característica para que tenga una media de cero y una desviación estándar de uno. Se calcula restando el valor medio y dividiendo el resultado por la desviación estándar. Aunque no limita los valores a un rango específico como la normalización, los mantiene en rangos similares.

Desequilibrio de datos (Data Imbalance)

En muchos problemas del mundo real, una clase es significativamente más frecuente que otra. Por ejemplo, en la detección de transacciones fraudulentas, hay muchas más transacciones legítimas (ejemplos negativos) que fraudulentas (ejemplos positivos). Esto puede llevar a que el modelo se adapte mejor a la clase mayoritaria y no sea capaz de predecir correctamente los ejemplos de la clase minoritaria. Para equilibrar el conjunto de datos, se pueden aplicar técnicas como:

- **Over-sampling:** Repetir los ejemplos de la clase minoritaria (por ejemplo, duplicar las transacciones fraudulentas).
- **Under-sampling:** Seleccionar un subconjunto de la clase mayoritaria (por ejemplo, desechar la mitad de los ejemplos legítimos).
- **Modificar la función de error:** Ajustar la función de error para que los ejemplos de la clase minoritaria tengan más influencia en la construcción del modelo.

Consideración general para todas las transformaciones

- Es crucial aplicar estas transformaciones solo al conjunto de datos de entrenamiento inicialmente. Posteriormente, las mismas transformaciones (con los parámetros aprendidos del conjunto de entrenamiento) deben aplicarse a los conjuntos de validación y prueba, así como a cualquier nuevo dato, para asegurar que el algoritmo reciba las características de entrada en el mismo formato en que fue entrenado.
- Los algoritmos de machine learning, por norma general, requieren que las características de entrada sean numéricas para poder funcionar y construir un modelo o ajustar una función hipótesis.

Caso Práctico: Creación de Pipelines y Transformadores personalizados

- Ver notebook 9_Creación de Transformadores y Pipelines personalizados_CA.ipynb
- Los transformadores personalizados simplifican los pasos complejos de preparación de datos.
- Los pipelines encadenan múltiples transformadores y estimadores para optimizar los flujos de trabajo.
- El `ColumnTransformer` permite la aplicación en paralelo de diferentes pipelines a diversos subconjuntos de datos.

Pipelines y Transformadores

- Los pipelines y transformadores personalizados en el aprendizaje automático resuelven el problema de la preparación tediosa y secuencial de los datos. En la mayoría de los conjuntos de datos reales, es necesario aplicar varias transformaciones, como manejar valores nulos, convertir valores categóricos a numéricos y estandarizar o normalizar datos con escalas diferentes. Realizar estas transformaciones una por una, resulta muy tedioso.

Pipelines y Transformadores

- Los pipelines y transformadores personalizados, especialmente en Scikit-learn, ofrecen una solución al permitir agrupar todas las operaciones de transformación necesarias en un único flujo de ejecución. Esto facilita las transformaciones para diferentes conjuntos de datos y permite aplicar de manera secuencial el método ``fit_transform`` a todos los transformadores, pasando la salida de uno como entrada al siguiente. Además, la creación de transformadores propios permite mantener un código más limpio y estructurado, facilitando la reutilización del código para otros proyectos.

Pipelines y Transformadores

La API de Scikit-learn facilita la creación de transformadores personalizados a través de su estructura de estimadores, transformadores y predictores. Un estimador es el objeto base que puede estimar cualquier parámetro y se construye utilizando el método ``fit``. A partir de los estimadores, se forman dos objetos fundamentales:

1. Transformadores: Son estimadores capaces de transformar conjuntos de datos, como convertir variables categóricas a numéricas o escalar un conjunto de datos. La transformación se realiza mediante el método ``transform``, que recibe un conjunto de datos como parámetro de entrada.
2. Predictores: Son estimadores capaces de realizar predicciones, utilizando fundamentalmente el método ``predict``, que recibe un conjunto de datos sin la etiqueta de salida y devuelve un nuevo conjunto de datos o una matriz de NumPy con las predicciones.

Pipelines y Transformadores

Para ello, Scikit-learn proporciona muchos transformadores ya implementados, pero también permite implementar transformadores propios de manera sencilla. Para crear un transformador personalizado, se debe crear una clase de Python que herede de las clases `BaseEstimator` y `TransformerMixin` de Scikit-learn. Esta clase debe implementar fundamentalmente dos métodos:

- `fit`: Para construir el transformador. En muchos casos, si el transformador no necesita aprender parámetros del conjunto de datos, este método puede no hacer nada.
- `transform`: Para aplicar la transformación al conjunto de datos.

Al seguir esta estructura, los transformadores personalizados se pueden invocar exactamente igual que los transformadores ya implementados en Scikit-learn, utilizando métodos como `fit_transform`. Esto permite mantener un código más limpio y estructurado al preparar los datos y facilita la reutilización del código para otros proyectos.

Pipelines y Transformadores

- `ColumnTransformer` permite agrupar un flujo de ejecución con todas las operaciones de transformación necesarias sobre el conjunto de datos. Esto facilita las transformaciones para diferentes conjuntos de datos.
- `ColumnTransformer` ejecuta todos los pipelines en paralelo y concatena el resultado de cada uno de los pipelines en un mismo conjunto de datos. Esto resuelve el problema de tener que separar características numéricas y categóricas, aplicar diferentes transformadores a cada una y luego volver a unir las manualmente.
- Con `ColumnTransformer`, se puede construir un pipeline para las características numéricas (que aplique escalado, elimine valores nulos, etc.) y otro pipeline para las características categóricas (que aplique, por ejemplo, un `OneHotEncoder`). `ColumnTransformer` ejecuta estos pipelines en paralelo y luego concatena los resultados en un único conjunto de datos.
- De esta manera, se puede transformar un conjunto de datos original con variables categóricas, valores nulos y variables numéricas en un conjunto de datos preprocesado que ha eliminado valores nulos, escalado valores numéricos y aplicado One Hot Encoding a los atributos categóricos, todo ello con un código más limpio y eficiente.

Caso Práctico: Evaluación de resultados

- Ver el notebook 10_Evaluación de resultados_CA.ipynb

El objetivo principal de utilizar métricas de evaluación en ML es verificar si el modelo funciona adecuadamente, después de que el conjunto de datos ha sido preprocesado y el algoritmo de ML ha sido entrenado para generar un modelo.

Estas métricas permiten comparar las predicciones del modelo con las etiquetas originales que se tienen en los conjuntos de validación y prueba, ya que no es posible realizar estas comparaciones de forma manual debido al gran volumen de datos.

Evaluación de resultados

- Aquí se explica cómo evaluar el rendimiento de los modelos de aprendizaje automático.
- Detalla métricas numéricas como precisión, exhaustividad y puntuación F1, derivadas de una matriz de confusión (verdaderos positivos, verdaderos negativos, falsos positivos y falsos negativos).
- También se analizan métricas gráficas, como las curvas ROC y PR, para visualizar el rendimiento del modelo.
- Se destaca la puntuación F1 como una métrica integral que combina precisión y exhaustividad.

Evaluación de resultados

Se define y diferencia los Verdaderos Positivos, Verdaderos Negativos, Falsos Positivos y Falsos Negativos utilizando la matriz de confusión de la siguiente manera:

- Verdadero Positivo (VP): Ocurre cuando el modelo predice que una transacción es fraudulenta (predicción de uno) y el valor real en el conjunto de datos etiquetado para esa transacción también es fraudulento. Es decir, el algoritmo ha acertado.
- Verdadero Negativo (VN): Se da cuando el algoritmo predice que una transacción es legítima y, efectivamente, en el conjunto de datos de validación estaba etiquetada como legítima. Es decir, el algoritmo ha acertado al clasificarla como no fraudulenta.
- Falso Positivo (FP): Se produce cuando el algoritmo predice que una transacción es fraudulenta, pero la etiqueta real en el conjunto de datos de validación indica que la transacción es legítima. Esto significa que el algoritmo detecta como fraudulenta una transacción que en realidad no lo es.
- Falso Negativo (FN): Sucede cuando el algoritmo predice que una transacción es legítima (predicción de cero), pero el valor real en el conjunto de datos de validación es fraudulento. En este caso, el algoritmo no logra identificar una transacción fraudulenta, dejándola pasar como legítima.

Estas definiciones se basan en la comparación entre el valor predicho por el modelo (por ejemplo, fraude o legítimo) y el valor real o la etiqueta que se tiene en el conjunto de datos de validación.

Evaluación de resultados

El F1 Score, la curva ROC y la curva PR son métricas y representaciones gráficas utilizadas para evaluar el rendimiento de un modelo de machine learning.

- F1 Score: El F1 Score se interpreta como un promedio ponderado de la precisión y la exhaustividad (recall). Su valor óptimo es 1 y el peor es 0. Esta métrica combina la visión de los falsos positivos (proporcionada por la precisión) y los falsos negativos (proporcionada por la exhaustividad), ofreciendo una visión conjunta y completa del rendimiento del algoritmo. Cuanto más cercano a 1 sea el F1 Score, mejor se comporta el algoritmo. Por ejemplo, un F1 Score de 0.9 podría interpretarse como que el algoritmo clasifica correctamente el 90% de los casos.
- Curva ROC (Receiver Operating Characteristic): La curva ROC representa la tasa de verdaderos positivos en el eje Y (vertical) y la tasa de falsos positivos en el eje X (horizontal). El punto ideal en esta gráfica es la esquina superior izquierda, que corresponde a una tasa de falsos positivos de cero y una tasa de verdaderos positivos de uno. Un área más grande debajo de la curva generalmente indica un mejor rendimiento del modelo.
- * Eje X (Tasa de Falsos Positivos): Se desea que sea baja, cercana a cero.
- Eje Y (Tasa de Verdaderos Positivos): Se desea que sea alta, cercana a uno.
- Un modelo óptimo tendría una curva ROC que llega a la esquina superior izquierda, lo que significa que ha clasificado correctamente el 100% de los casos, sin falsos positivos. La curva ROC ayuda a visualizar cómo se comporta el modelo al variar el umbral de clasificación, permitiendo comparar el rendimiento de diferentes modelos.